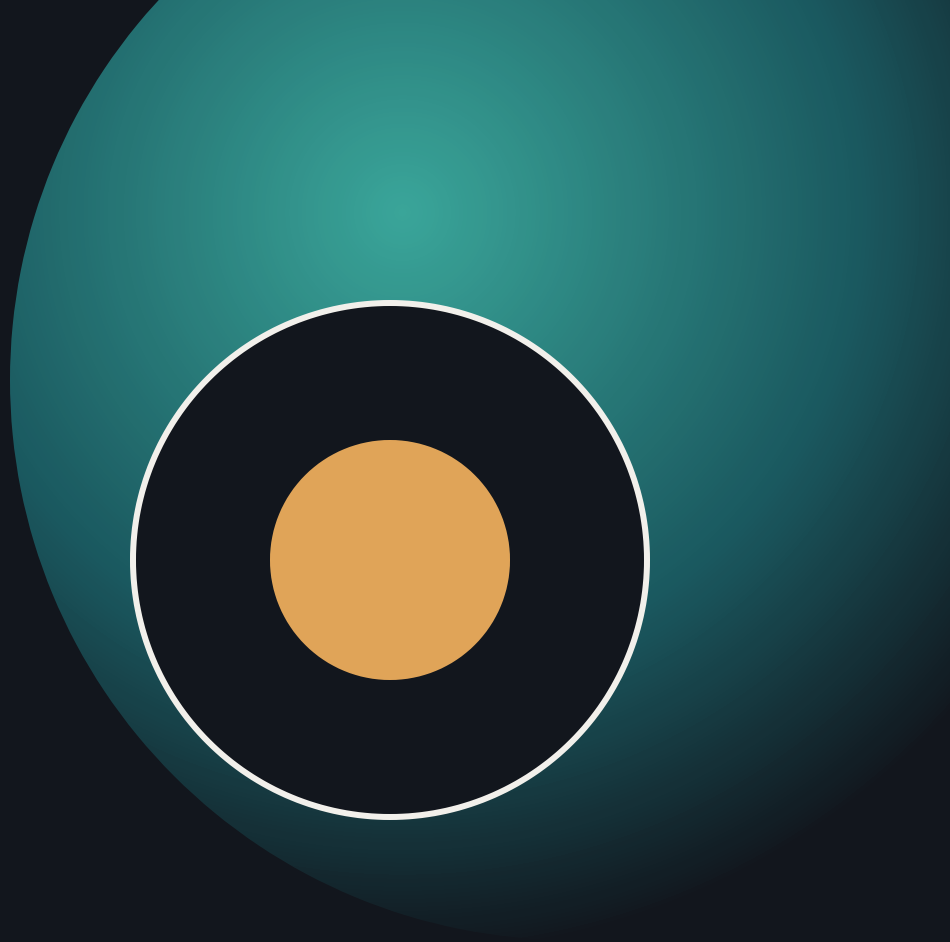
PROJECT PRESENTATION

Third Eye

A phone-based referee for physical Jenga: LiDAR depth, marker pose and a compiled rulebook



Aadit Pani
1RVU23CSE002

Mitul KM
1RVU23CSE273

Megh Mavani
1RVU23CSE260

THE PROBLEM

Who watches the tower?

A physical game needs a referee who knows where every piece is and what the rulebook says. For Jenga that is a tower of layers of three blocks, and players cannot always agree on what just happened.

1

Which block was pulled?

Name the layer and slot, even though the tower looks the same from every side.

2

What may be pulled next?

Rules and balance: never the top layers, never a block that would topple the tower.

3

Did it fall?

Call the game over, without being fooled by a hand in front of the camera.

Constraints: 15 mm layers, a 75 mm wide tower, a few seconds between human actions, and only consumer phones.

OUR APPROACH

Two phones, four markers, one rulebook

An iPhone with LiDAR and an Android phone watch the tower. Printed markers fix the tower in space; the rules decide what is legal. The player stays in control of the final answer.

2

phones: iPhone LiDAR plus Android camera, over Wi-Fi or a USB cable

4

ArUco markers on a fixed board, so the tower frame is the same every game

90%

confidence needed before a pulled block is added automatically; the player answers Yes or No

0

extra hardware: no force sensors, no special table

We build on existing parts (ArUco, ARKit, MediaPipe) and contribute the integration, the calibration and the guards that make it dependable in use.

SYSTEM ARCHITECTURE

From camera frame to rule verdict

iPhone (LiDAR)
ARKit frames, 256×192 depth, pose

Android phone
720p JPEG stream

→

Game server (Mac)

marker pose · LiDAR tower fit · pull finder · fall watch · pull ledger · shape list for the overlay

→

Third Eye server

follows the ledger · compiled rulebook checker · balance model · collapse · LLM referee

→

Web app

Game · 3D tower · Replay

Wi-Fi by default

The phones post frames over HTTP. Nothing has to be installed on the Mac beyond the servers.

USB when plugged in

A relay on the iPhone and *adb reverse* on Android switch to the cable on their own, at a higher frame rate.

Player in the loop

MARK after each pull, then Yes or No. The ledger of confirmed pulls is trusted over noisy depth.

FRAME ACQUISITION

One callback, three consumers

ARKit delivers about 60 frames a second. We keep only what each stage needs, and drop frames instead of queueing them.

P **Preview**

960 px JPEG, every 6th frame on Wi-Fi (every 2nd on USB), with a limit on requests in flight.

D **Depth**

256×192 depth and confidence maps, about twice a second, for the tower fit and the fall check.

R **Recording**

Every 20th good frame: full-resolution JPEG, depth, intrinsics and pose.

Measured save rate, three real games (Hz)



Median spacing 333 ms in every game, matching 60 Hz ÷ 20. The shortfall is two gaps of 5.2 s and 3.6 s, probably tracking pauses. 1,706 frames, none missing a file.

0.8 MB
per saved frame

133–153
MB per minute

OVERLAY ACCURACY

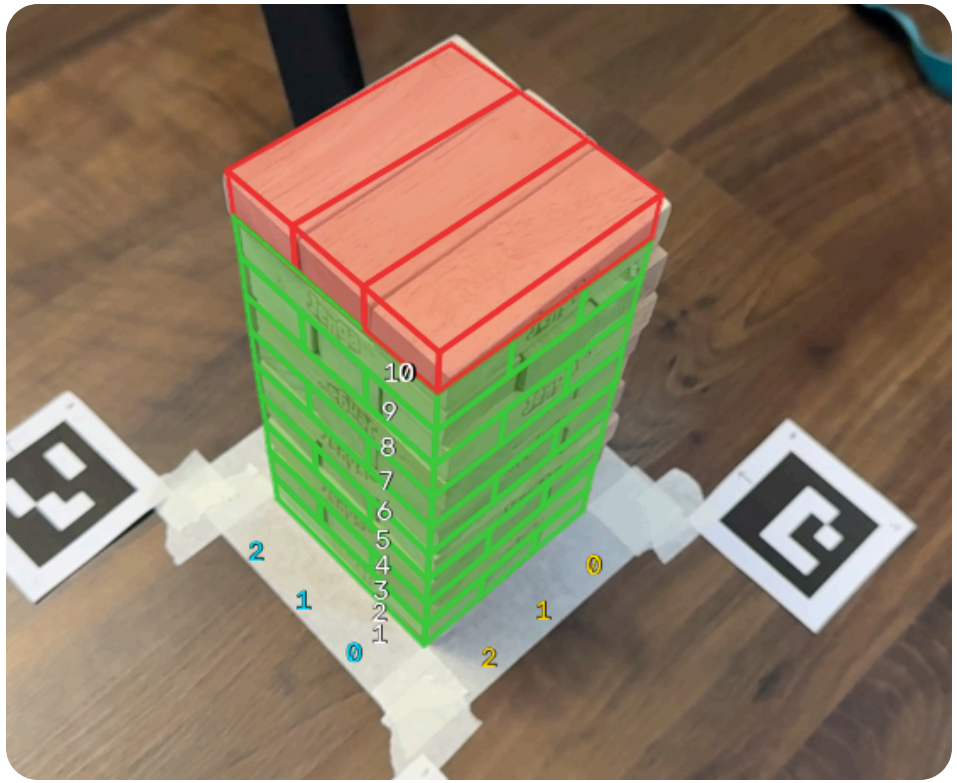
Making the overlay sit on the tower

1 Fixed, calibrated board
The markers never move, so their exact layout is solved once by bundle adjustment and reused every game.

2 LiDAR correction
A robust 7-parameter box fit nudges the iPhone pose. The Android has no depth, so it must not get that correction (it sat one layer too high).

3 Edge-snapping per camera
The projected outline is nudged onto the tower's real image edges, redone when a phone moves.

4 Smoothing and lock
EMA pose smoothing, pose reuse when the markers are still, and a Lock button once it looks right.



Layers counted from the bottom; green blocks may be pulled, red may not.

Measured: marker error 1.1–1.6 px; snapping shifts the pose by at most 2.4 mm and 0.8° and about doubles the edge-alignment score (5 Android frames).

PULL IDENTIFICATION

Naming the pulled block

Depth alone cannot see a block leave: a 15–35 mm dent is close to the sensor noise. So we compare how each layer's end face looks before and after the move.

1

MARK

Player taps after the pull and moves the hand away.

→

2

Calm windows

Find still moments before and after, using hand and motion activity.

→

3

Compare faces

32×32 crops per layer and camera, z-scored and summed over both cameras.

→

4

Confidence

Calibrated tiers from the score and its gap to the runner-up.

90% or more

The block is added to the tower. The player answers **Yes** or **No** (keys Y and N).

below 90%

Nothing is added. The player types the move, for example **l 5**, or **fell**.

RESULTS

How often is the finder right?

31 pulls with a known layer, 10 sessions



Honest caveat: a confident answer the player did not correct counts as right, and the rules were tuned on these same games. These are not held-out figures.

5 of 5

moves answered at 95% and applied automatically in one 8-layer game replayed on the calibrated board

0.2–0.3 s

of finder computation per move on a Mac, plus a calm window of at least 1.6 s (it was 4.5 s)

Old games, recorded before the board was calibrated, fail when replayed. The finder depends on the calibrated board.

RULES AND THE REFEREE

What may be pulled, and why

green: may be pulled | amber: loose | red: not allowed

Balance rule

After a hypothetical pull, the centre of mass of everything above each layer must stay inside the footprint of the blocks left in it, with a 4 mm margin. The bottom layer and the top complete layer are never pullable.

Pulled blocks go back on top

The tower is modelled as the effective tower, so height and the top layer follow every move. A flag turns this off.

Compiled rulebooks

A natural-language rulebook is compiled into a validated spec that the checker evaluates against the block lattice.

15 / 16

corpus entries compiled with a 120B hosted model (9 of 16 failed with an 8B local model)

4 / 80

claims unsupported in generated reports (5.0%)

Ask the referee

An LLM answers questions such as *Which blocks can I still take safely?* from what the cameras saw and the checker decided, citing the rules.

COLLAPSE DETECTION

Calling the fall, and never a false one

Two checks, each corroborated

A height watch on the fitted box and a standing check on the point cloud. Each needs the 95th-percentile height to drop by at least 30 mm.

Failures that taught us the guards

Hand over the lens
Compared with a baseline taken while the tower stood.

Height collapse after top pulls
Trust the fit height, corroborate with p95.

No points at all
Handled as a covered lens, not a fall.

Banner stuck on
A false alarm clears itself on the next readings.

1 / 1

real falls flagged (381 s into the run) in three replayed games

0

false alarms in the other two games (66 and 99 depth frames)

Weak evidence: the guards were designed on these games. A real hand over the lens was tested by blanking depth in a replay.

THE INTERFACE

Everything on one screen

Game

Both camera feeds with the red, green and amber overlay, plus every control: record, MARK, Yes or No, type the move, tower settings, re-detect, lock, rotate view, new game.

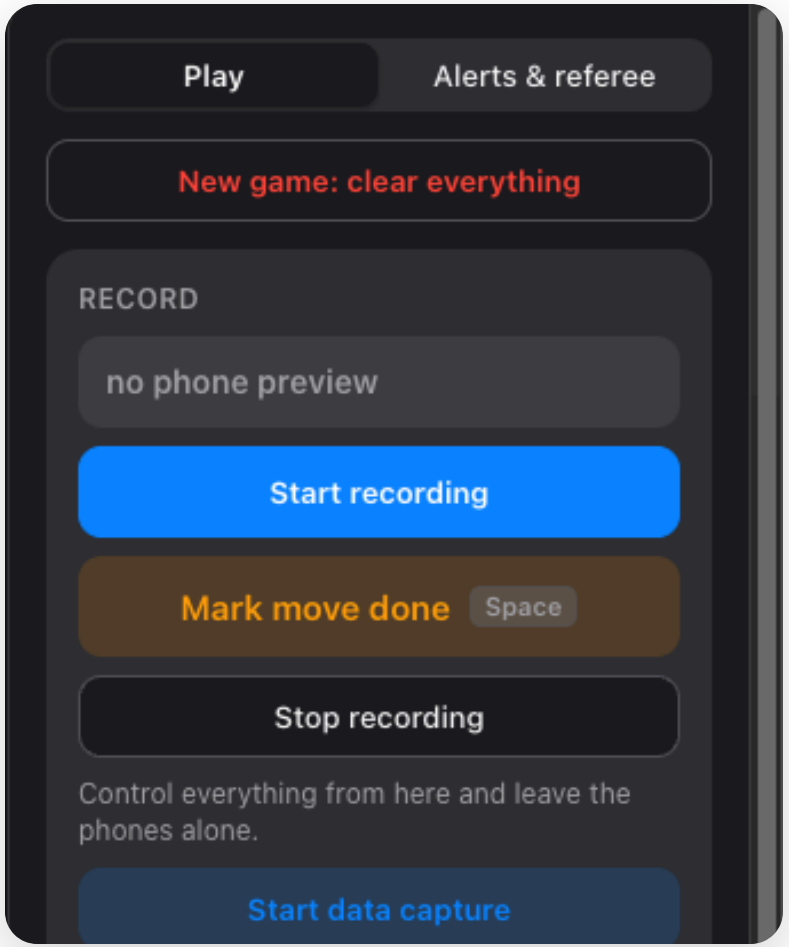
3D tower

The same controls beside a live 3D tower. When you click Yes the block comes off the model in about half a second, and the rules, alerts and referee follow along.

Replay

Rebuilds any recorded game move by move from the event log, with a 3D scrubber.

Drawn in the browser on the GPU: the server sends a raw JPEG and a short list of shapes, so labels stay upright when a view is rotated.



PERFORMANCE

Where the time goes

Median stage cost, ms (laptop CPU, one thread)



The robust fit dominates the depth path (about 67 ms) but runs at roughly 2 Hz, about 13% of one core. The rule checker grows as L^3 (measured slope 2.80).

8–10×

faster than real time: the full pipeline replays at 24–28 ms per frame against a 333 ms frame period

0.3 s

of finder computation per move on the Mac

LIMITS AND NEXT STEPS

What we have not shown yet

Limits

- Small data: three games for acquisition, 31 labelled pulls, three replays for falls; one tower and one iPhone model.
- Rules and guards were tuned on the same games, so nothing is held out.
- Below 90% the player still types the layer; placements are assumed.
- Wired versus Wi-Fi frame rate and end-to-end latency were not measured.

Next

- Overlay error against hand-labelled tower corners, before and after calibration.
- A held-out finder test on new games with independent ground truth.
- Measured time from MARK to the answer on the player's screen.
- More recorded falls, including a real hand over the lens.

THANK YOU

Questions?

Third Eye: a phone-based referee for physical Jenga.

Aadit Pani
1RVU23CSE002

Mitul KM
1RVU23CSE273

Megh Mavani
1RVU23CSE260